



Multi-layer Intelligent Evaluation for Smart Contracts

Smart Contract Security Audit

FHE-ML Platform Contracts V2

Xcapit

Prepared by: **Fernando Boiero - MIESC Security**

Audit Date: 2026-01-24

Report Date: **2026-01-24 23:31:27**

Version: 1.0

CONFIDENTIAL

This document contains confidential security findings and is intended solely for the addressee. Unauthorized distribution is prohibited.

Table of Contents

1. Executive Summary
 2. Scope & Methodology
 3. Risk Assessment
 4. Findings Overview
 5. Detailed Findings
 6. Remediation Roadmap
 7. Appendices
-

1. Executive Summary

1.1 Key Takeaways

In this smart contract audit report, we have identified 15 issues, none of which pose critical or high risks to your organization. However, it's essential to address the low-level concerns to ensure the long-term security and reliability of your blockchain application.

The most significant risk pertains to missing zero-checks, which could potentially allow an attacker to manipulate transactions and cause financial losses. Although the likelihood is relatively low, the potential impact on your business could be substantial, particularly in high-value transactions.

Additionally, we found several assembly-related issues that may not pose immediate risks but could lead to unintended consequences or inefficiencies in the contract's execution.

To ensure deployment readiness, we recommend addressing these low-level concerns before proceeding with the implementation of this smart contract. By doing so, you can minimize potential financial losses and maintain a strong security posture for your blockchain application.

In summary, while the overall risk level is relatively low, it's crucial to address the identified issues to ensure the long-term success and security of your blockchain initiative.

1.2 Deployment Recommendation

Recommendation: **GO**

The audit results show no critical or high severity vulnerabilities, and only a few low severity issues. These can be addressed post-deployment as they do not pose an immediate threat to the system.

1.3 Risk Summary

METRIC	VALUE
Overall Risk Score	21/100
Exploitability	Medium
Business Impact	Medium
Confidence Level	High

Findings by Severity

SEVERITY	COUNT	% OF TOTAL
Critical	0	0.0%
High	0	0.0%
Medium	0	0.0%
Low	3	20.0%
Informational	12	80.0%
Total	15	100%

1.4 Estimated Value at Risk

Value at risk estimation requires additional context about contract TVL and usage patterns.

2. Scope & Methodology

2.1 Engagement Details

PROPERTY	VALUE
Client	Xcapit
Contract	FHE-ML Platform Contracts V2
Repository	github.com/Xcapit/FHE-ML-Platform
Commit Hash	N/A
Network	Ethereum Mainnet
Engagement Type	Security Audit

2.2 Scope

In Scope

FILE	LINES	DESCRIPTION
------	-------	-------------

Total: 3 files, N/A lines of code

Out of Scope

- External dependencies and imported libraries
- Off-chain components
- Economic/tokenomics analysis
- Frontend/backend applications

2.3 Methodology

This audit employed MIESC's comprehensive 9-layer defense-in-depth methodology:

Layer 1: Static Analysis	[--]
Layer 2: Dynamic Testing	[--]
Layer 3: Symbolic Execution	[--]
Layer 4: Formal Verification	[--]
Layer 5: Property Testing	[--]
Layer 6: AI/LLM Analysis	[--]
Layer 7: Pattern Recognition	[--]
Layer 8: DeFi Security	[--]
Layer 9: Advanced Detection	[--]

Tools Utilized

LAYER	TOOL	VERSION	STATUS
-------	------	---------	--------

Audit Process

- 1. **Initial Assessment** - Review documentation, understand architecture
- 2. **Automated Analysis** - Execute multi-layer tool suite
- 3. **Manual Review** - Deep dive into flagged code sections
- 4. **AI Correlation** - Cross-reference findings, reduce false positives
- 5. **Verification** - Reproduce and validate vulnerabilities
- 6. **Report Generation** - Document findings with remediation guidance

2.4 Limitations

- Time-boxed engagement (N/A)
- Analysis based on code snapshot at commit N/A
- No guarantee of finding all vulnerabilities
- Economic attack vectors not fully modeled
- Dependency vulnerabilities may exist beyond analysis scope

3. Risk Assessment

3.1 Risk Matrix

The following matrix maps findings by **Impact** (vertical) and **Likelihood** (horizontal):

		LIKELIHOOD		
		Low	Medium	High
IMPACT →	High	Medium 0	High 0	Critical 0
	Med	Low 0	Medium 0	High 0
	Low	Info 12	Low 3	Medium 0

3.2 CVSS-like Scoring

FINDING ID	TITLE	BASE SCORE	VECTOR
slither-missing-zero-check-0	missing-zero-check	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-assembly-1	assembly	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-assembly-2	assembly	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-assembly-3	assembly	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-assembly-4	assembly	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-missing-zero-check-0	missing-zero-check	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-assembly-1	assembly	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-assembly-2	assembly	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-assembly-3	assembly	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-assembly-4	assembly	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-missing-zero-check-0	missing-zero-check	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-assembly-1	assembly	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-assembly-2	assembly	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-assembly-3	assembly	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N
slither-assembly-4	assembly	3.1	AV:N/AC:H/PR:L/UI:N/C:N/I:L/A:N

Scoring Methodology: - **Attack Vector (AV):** Network, Adjacent, Local, Physical - **Attack Complexity (AC):** Low, High - **Privileges Required (PR):** None, Low, High - **User Interaction (UI):** None, Required - **Impact:** Confidentiality, Integrity, Availability

3.3 Risk Narrative

In the recent audit, we observed a concentration of issues primarily categorized under 'assembly' and 'missing-zero-check'. This finding indicates potential vulnerabilities in the contract's security architecture, specifically related to the assembly code and lack of proper zero-value checks.

The primary attack vectors could be code injection attacks or integer overflows/underflows due to the assembly issues. Meanwhile, missing zero-checks might lead to unintended function behavior or data corruption, potentially enabling an attacker to manipulate contract state.

Given these findings, the likelihood of exploitation is considered moderate to high, as attackers may leverage these vulnerabilities for malicious activities such as draining funds, altering contract state, or executing unauthorized actions.

Immediate actions should focus on addressing the assembly issues by thoroughly reviewing and securing the assembly code. Additionally, implementing proper zero-checks can help prevent data corruption and unintended function behavior. It is essential to prioritize these fixes to minimize potential risks and ensure the contract's security posture remains robust.

4. Findings Overview

4.1 Summary Table

ID	TITLE	SEVERITY	STATUS	CVSS
F-001	missing-zero-check	LOW	open	3.1
F-002	assembly	INFO	open	3.1
F-003	assembly	INFO	open	3.1
F-004	assembly	INFO	open	3.1
F-005	assembly	INFO	open	3.1
F-006	missing-zero-check	LOW	open	3.1
F-007	assembly	INFO	open	3.1
F-008	assembly	INFO	open	3.1
F-009	assembly	INFO	open	3.1
F-010	assembly	INFO	open	3.1
F-011	missing-zero-check	LOW	open	3.1
F-012	assembly	INFO	open	3.1
F-013	assembly	INFO	open	3.1
F-014	assembly	INFO	open	3.1
F-015	assembly	INFO	open	3.1

4.2 Category Distribution

CATEGORY	COUNT	SEVERITY BREAKDOWN
assembly	12	12 info
missing-zero-check	3	3 low

4.3 Layer Coverage Analysis

LAYER	TOOLS RUN	PASSED	FAILED	FINDINGS	COVERAGE

5. Detailed Findings

F-001. missing-zero-check

PROPERTY	VALUE
Severity	LOW
Category	missing-zero-check
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/access/Ownable2Step.sol:43 (newOwner)
Status	open
Detected By	slither

Description

Ownable2Step.transferOwnership(address).newOwner lacks a zero-check on : -
_pendingOwner = newOwner

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Minor impact on contract functionality or gas efficiency.

Recommendation

Review and fix the vulnerability

References

- No references available

F-002. assembly

PROPERTY	VALUE
Severity	INFO
Category	assembly
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/utils/StorageSlot.sol:66 (getAddressSlot)
Status	open
Detected By	slither

Description

StorageSlot.getAddressSlot(bytes32) uses assembly - INLINE ASM

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Informational finding for code quality improvement.

Recommendation

Review and fix the vulnerability

References

- No references available

F-003. assembly

PROPERTY	VALUE
Severity	INFO
Category	assembly
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/utils/StorageSlot.sol:102 (getInt256Slot)
Status	open
Detected By	slither

Description

StorageSlot.getInt256Slot(bytes32) uses assembly - INLINE ASM

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Informational finding for code quality improvement.

Recommendation

Review and fix the vulnerability

References

- No references available

F-004. assembly

PROPERTY	VALUE
Severity	INFO
Category	assembly
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/utils/StorageSlot.sol:129 (getBytesSlot)
Status	open
Detected By	slither

Description

StorageSlot.getBytesSlot(bytes32) uses assembly - INLINE ASM

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Informational finding for code quality improvement.

Recommendation

Review and fix the vulnerability

References

- No references available

F-005. assembly

PROPERTY	VALUE
Severity	INFO
Category	assembly
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/utils/StorageSlot.sol:120 (getStringSlot)
Status	open
Detected By	slither

Description

StorageSlot.getStringSlot(string) uses assembly - INLINE ASM

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Informational finding for code quality improvement.

Recommendation

Review and fix the vulnerability

References

- No references available

F-006. missing-zero-check

PROPERTY	VALUE
Severity	LOW
Category	missing-zero-check
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/access/Ownable2Step.sol:43 (newOwner)
Status	open
Detected By	slither

Description

Ownable2Step.transferOwnership(address).newOwner lacks a zero-check on : -
_pendingOwner = newOwner

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Minor impact on contract functionality or gas efficiency.

Recommendation

Review and fix the vulnerability

References

- No references available

F-007. assembly

PROPERTY	VALUE
Severity	INFO
Category	assembly
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/utils/StorageSlot.sol:66 (getAddressSlot)
Status	open
Detected By	slither

Description

StorageSlot.getAddressSlot(bytes32) uses assembly - INLINE ASM

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Informational finding for code quality improvement.

Recommendation

Review and fix the vulnerability

References

- No references available

F-008. assembly

PROPERTY	VALUE
Severity	INFO
Category	assembly
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/utils/StorageSlot.sol:102 (getInt256Slot)
Status	open
Detected By	slither

Description

StorageSlot.getInt256Slot(bytes32) uses assembly - INLINE ASM

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Informational finding for code quality improvement.

Recommendation

Review and fix the vulnerability

References

- No references available

F-009. assembly

PROPERTY	VALUE
Severity	INFO
Category	assembly
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/utils/StorageSlot.sol:129 (getBytesSlot)
Status	open
Detected By	slither

Description

StorageSlot.getBytesSlot(bytes32) uses assembly - INLINE ASM

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Informational finding for code quality improvement.

Recommendation

Review and fix the vulnerability

References

- No references available

F-010. assembly

PROPERTY	VALUE
Severity	INFO
Category	assembly
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/utils/StorageSlot.sol:120 (getStringSlot)
Status	open
Detected By	slither

Description

StorageSlot.getStringSlot(string) uses assembly - INLINE ASM

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Informational finding for code quality improvement.

Recommendation

Review and fix the vulnerability

References

- No references available

F-011. missing-zero-check

PROPERTY	VALUE
Severity	LOW
Category	missing-zero-check
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/access/Ownable2Step.sol:43 (newOwner)
Status	open
Detected By	slither

Description

Ownable2Step.transferOwnership(address).newOwner lacks a zero-check on : -
_pendingOwner = newOwner

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Minor impact on contract functionality or gas efficiency.

Recommendation

Review and fix the vulnerability

References

- No references available

F-012. assembly

PROPERTY	VALUE
Severity	INFO
Category	assembly
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/utils/StorageSlot.sol:66 (getAddressSlot)
Status	open
Detected By	slither

Description

StorageSlot.getAddressSlot(bytes32) uses assembly - INLINE ASM

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Informational finding for code quality improvement.

Recommendation

Review and fix the vulnerability

References

- No references available

F-013. assembly

PROPERTY	VALUE
Severity	INFO
Category	assembly
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/utils/StorageSlot.sol:102 (getInt256Slot)
Status	open
Detected By	slither

Description

StorageSlot.getInt256Slot(bytes32) uses assembly - INLINE ASM

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Informational finding for code quality improvement.

Recommendation

Review and fix the vulnerability

References

- No references available

F-014. assembly

PROPERTY	VALUE
Severity	INFO
Category	assembly
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/utils/StorageSlot.sol:129 (getBytesSlot)
Status	open
Detected By	slither

Description

StorageSlot.getBytesSlot(bytes32) uses assembly - INLINE ASM

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Informational finding for code quality improvement.

Recommendation

Review and fix the vulnerability

References

- No references available

F-015. assembly

PROPERTY	VALUE
Severity	INFO
Category	assembly
CVSS Score	3.1
Location	lib/openzeppelin-contracts/contracts/utils/StorageSlot.sol:120 (getStringSlot)
Status	open
Detected By	slither

Description

StorageSlot.getStringSlot(string) uses assembly - INLINE ASM

Vulnerable Code

```
// No code snippet
```

Impact Analysis

Informational finding for code quality improvement.

Recommendation

Review and fix the vulnerability

References

- No references available

6. Remediation Roadmap

6.1 Prioritized Actions

PRIORITY	FINDING	SEVERITY	EFFORT	RATIONALE
1	missing-zero-check	Low	Medium	The missing-zero-check vulnerability in <code>Ownable2Step.transferOwnership(address).newOwner</code> could potentially allow an attacker to transfer ownership with zero as the new owner, causing unintended consequences. This issue has a high exploitability and business impact.
2	assembly	Info	Medium	The assembly function <code>StorageSlot.getAddressSlot(bytes32)</code> is marked as INFO, but it could potentially be used to leak storage addresses, which might lead to information disclosure or further exploitation. Although the business impact is lower compared to the first priority issue, it still requires attention.
3	missing-zero-check	Low	Medium	The duplicate missing-zero-check vulnerability in <code>Ownable2Step.transferOwnership(address).newOwner</code> has a lower exploitability and business impact compared to the first priority issue, but it still requires fixing for code consistency.
4	assembly	Info	Medium	The assembly function <code>StorageSlot.getInt256Slot(bytes32)</code> is marked as INFO and might not pose a significant threat if proper input validation is in place. However, it's recommended to review this function for potential issues.
5	assembly	Info	Medium	The assembly function <code>StorageSlot.getBytesSlot(bytes32)</code> has a similar risk profile as <code>StorageSlot.getInt256Slot(bytes32)</code> , but it's recommended to review this function for potential issues.
6	assembly	Info	Medium	The assembly function <code>StorageSlot.getStringSlot(string)</code> has a lower risk compared to the previous findings, as it deals with strings. However, it's still recommended to review this function for potential issues.
7	assembly	Info	Medium	The assembly function <code>StorageSlot.getAddressSlot(bytes32)</code> is marked as INFO and might not pose a significant threat if proper input validation is in place. However, it's recommended to review this function for potential issues.
8	assembly	Info	Medium	The assembly function <code>StorageSlot.getInt256Slot(bytes32)</code> has a similar risk profile as <code>StorageSlot.getAddressSlot(bytes32)</code> , but

PRIORITY	FINDING	SEVERITY	EFFORT	RATIONALE
				it's recommended to review this function for potential issues.
9	assembly	Info	Medium	The assembly function <code>StorageSlot.getBytesSlot(bytes32)</code> has a similar risk profile as <code>StorageSlot.getInt256Slot(bytes32)</code> , but it's recommended to review this function for potential issues.

6.2 Remediation Timeline

PHASE	WEEK	PRIORITY	FINDINGS	ACTION
1	Week 1	Critical & High	0 + 0	Immediate remediation required
2	Week 2	Medium	0	Address medium severity issues
3	Week 3	Low & Info	3 + 12	Fix low priority items
4	Week 4	Verification	-	Re-audit and validation

6.3 Quick Wins

Review findings marked as "Low" effort for quick security improvements.

6.4 Effort vs Impact Matrix

		IMPACT		
		Low	Medium	High
	High	Avoid	Consider	Schedule
	Med	Defer	Plan	Priority
	Low	If Time	Quick Win	DO FIRST!
		EFFORT →		

7. Appendices

Appendix A: Tool Execution Details

Appendix B: Files Analyzed

#	FILE PATH	LINES	FUNCTIONS	FINDINGS
1	Unknown	N/A	--	15

Appendix C: SWC Registry Compliance

SWC ID	TITLE	STATUS	FINDING(S)
--------	-------	--------	------------

Appendix D: OWASP Smart Contract Top 10

RANK	CATEGORY	STATUS	FINDINGS
------	----------	--------	----------

Appendix E: Glossary

TERM	DEFINITION
Reentrancy	A vulnerability where an external call allows execution to re-enter the calling contract before the first execution completes
Integer Overflow	When an arithmetic operation results in a value larger than can be stored in the variable
Front-running	Exploiting knowledge of pending transactions to gain an advantage
Flash Loan Attack	Using uncollateralized loans within a single transaction to manipulate protocols
Oracle Manipulation	Attacking price feeds or external data sources to influence contract behavior
Access Control	Mechanisms that restrict who can execute sensitive functions

Appendix F: Audit Trail

EVENT	TIMESTAMP	HASH
Contract Snapshot	2026-01-24	N/A
Analysis Started	2026-01-24	--
Analysis Completed	2026-01-24 23:31:27	--
Report Generated	2026-01-24 23:31:27	--
Report Hash	2026-01-24 23:31:27	N/A

Disclaimer

This audit report is provided on an "AS IS" basis without warranties of any kind, whether express or implied. The findings and recommendations represent the auditor's professional opinion based on the scope and methodology described.

Limitations: - This audit does not guarantee the absence of vulnerabilities - Smart contract security is an evolving field - New vulnerabilities may be discovered post-audit - Economic and governance attacks may not be fully modeled - The client is responsible for implementing and testing fixes

AI Disclosure: Sections labeled as "AI-Generated" or "AI Analysis" were produced using large language models (mistral:latest). These AI-generated insights are supplementary and should be reviewed by qualified security professionals. AI outputs may contain inaccuracies and are provided for informational purposes only.

Powered by [MIESC](<https://github.com/fboiero/MIESC>) Multi-layer Intelligent Evaluation for Smart Contracts *Report generated: 2026-01-24 23:31:27* *Report Version: 1.0*